REAL-TIME DOCKER → AMAZON ECR → KUBERNETES

Production DevOps Project Documentation

React + FastAPI + PostgreSQL | Docker | Trivy | Amazon ECR | Kubernetes / AKS | CI/CD

Hands-on implementation guide generated from the project package.

1. Executive Summary

This project demonstrates a realistic container delivery lifecycle: application code is built into production Docker images, scanned for vulnerabilities, pushed to Amazon ECR, and deployed to Kubernetes/AKS with replicas, health checks, resource limits, rolling updates, HPA and NetworkPolicy.

2. Architecture

Internet → Ingress → Frontend Service → React/Nginx Pods → Backend Service → FastAPI Pods → PostgreSQL. CI/CD runs Git Push → Docker Build → Trivy Scan → ECR Push → Kubernetes Deployment.

3. Docker Design

The frontend uses a multi-stage Node build and an unprivileged Nginx runtime. The backend uses Python slim, installs pinned dependencies, creates UID 10001 and exposes a health check. .dockerignore files keep unnecessary files out of the build context.

4. Tagging Strategy

Use semantic release tags such as 1.0.0, commit tags such as sha-a1b2c3d4e5f6, and latest only for development convenience. Production Kubernetes deployments should reference immutable tags or image digests.

5. Security

Trivy gates HIGH/CRITICAL vulnerabilities before push. ECR scanning provides registry-side vulnerability visibility. Repository IAM should follow least privilege. Secrets must never be stored in Dockerfiles, Git or image layers.

6. Amazon ECR

Create separate frontend and backend repositories. Authenticate Docker with aws ecr get-login-password. Push version and commit-SHA tags. Apply lifecycle policies to control storage growth. For production, prefer immutable tags and a reviewed lifecycle policy.

7. Kubernetes / AKS

Kubernetes Deployments run three replicas for frontend and backend. Services provide stable discovery. Readiness and liveness probes protect traffic. HPA scales backend from 3 to 8 replicas. NetworkPolicy limits backend communication.

8. ECR Pull Authentication

The lab uses a Kubernetes imagePullSecret created from an ECR authorization token. ECR authorization tokens expire, so the secret must be refreshed. For production AKS, use an approved short-lived credential automation or an enterprise registry/identity architecture instead of long-lived AWS access keys in the cluster.

9. CI/CD

The Azure DevOps example verifies tooling, calculates the account/commit identity, logs in to ECR, builds images, runs Trivy, then pushes version and SHA tags. A GitHub Actions example using AWS OIDC is also

included.

10. Troubleshooting

ImagePullBackOff and ErrImagePull: inspect pod events and validate repository/tag/authentication. CrashLoopBackOff: inspect current and previous container logs. 502/503: validate services/endpoints/Ingress. OOMKilled: inspect resource usage and limits. ECR push failures: verify IAM identity and repository permissions.

11. Operational Runbook

Validate rollout status, pod readiness, service endpoints, HPA metrics, events and logs after each deployment. Use rollout history and rollout undo for safe rollback.

12. Interview Summary

The candidate should be able to explain why containers are built with minimal images, why images are scanned, why commit-SHA tags are traceable, how ECR authentication works, how Kubernetes performs rolling updates, and how to troubleshoot image-pull and runtime failures.

Command Reference

```
# Build
docker build -f app/backend/Dockerfile -t k8s-docker-ecr/backend:1.0.0 app/backend
docker build -f app/frontend/Dockerfile -t k8s-docker-ecr/frontend:1.0.0 app/frontend

# ECR login
aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin $REGISTRY

# Kubernetes
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/backend.yaml
kubectl apply -f k8s/frontend.yaml
kubectl -n k8s-ecr-app rollout status deployment/backend

# Troubleshooting
kubectl -n k8s-ecr-app describe pod <pod>
kubectl -n k8s-ecr-app logs <pod> --previous
kubectl -n k8s-ecr-app get events --sort-by=.lastTimestamp
```

Included files

Area	Key files
Application	app/frontend, app/backend
Docker	Dockerfiles, .dockerignore, Compose
Registry	Terraform ECR, ECR scripts, lifecycle policy
Kubernetes	Deployments, Services, ConfigMap, Secrets, HPA, Ingress, NetworkPolicy
CI/CD	Azure Pipelines + GitHub Actions
Security	Trivy config, IAM policy example
Docs	README, Architecture, Deployment, Troubleshooting